

ShoeBox

Rapport technique de l'Architecture et des choix méthodologiques

RPG Dev Challenge · Spring 2026

Par Ovide KUICHUA

1. Présentation du projet

Shoebox est un gestionnaire financier local conçu pour les travailleurs autonomes. Il ingère des documents financiers bruts (relevés bancaires, reçus, factures, notes) les transforme en transactions structurées et les présente dans un tableau de bord interactif. L'ensemble de la chaîne, de l'extraction de données à la visualisation, fonctionne en local sans dépendance à des services cloud payants.

CoucheCouche	TechnologieTechnologie
API backend	FastAPI ; uvicorn ; Pydantic; uvicorn ;
Base de données	SQLite (WAL mode); SQLAlchemy 2; SQLAlchemy; IDs préfixés
Parsing	PyMuPDF; pytesseract + Pillow; openpyxl; parser TXT algo manuel
Frontend	Dash; Plotly; DM Sans / DM Mono / Playfair Display
Tests & qualité	Pytest; SAVEPOINT isolation ; ruff

2. Pipeline backend — ingestion de fichiers

Chaque fichier téléversé via POST /files/upload traverse les étapes suivantes. Un échec sur un fichier n'interrompt pas le traitement des autres.

1	Validation du format Le routeur vérifie que l'extension du fichier correspond au DocType déclaré (.pdf/.xlsx pour INV, .txt pour NOTE, etc.). Les fichiers invalides sont rejetés immédiatement avec HTTP 422.	API
----------	--	------------

2	Résolution de la source de paiement L'IngestionService vérifie si le label de source existe déjà. Sinon, un nouveau PaymentSource est créé. C'est ainsi que les nouvelles cartes apparaissent automatiquement dans la vue Sources.	Services
3	Stockage physique du fichier DiskFileStorage sauvegarde les octets bruts sous data/uploads/ avec un nom préfixé SHA-256 pour la déduplication. Un même fichier uploadé deux fois n'est écrit qu'une seule fois sur disque.	Infra
4	Enregistrement UploadedFile Une ligne est insérée dans uploaded_files (nom, type, chemin, source_id). C'est le lien de traçabilité permanent entre un fichier physique et ses transactions extraites.	Infra / DB
5	Sélection du parser Le service itère sur les parsers enregistrés (PDF, Image/OCR, XLSX, TXT) en appelant can_parse(). Le premier qui correspond est utilisé. Principe OCP : ajouter un format = ajouter une classe.	Services
6	Extraction & catégorisation Le parser lit le fichier et retourne une liste d'objets Transaction (dataclasses purs). Chaque description est passée dans le moteur de catégorisation à base de règles. Les parsers OCR calculent un score de confiance (0.0–1.0).	Infra
7	Validation métier Chaque transaction est validée : montant nul, date future, cohérence TVQ/TPS pour les reçus officiels. Les transactions invalides sont marquées is_flagged=True plutôt que rejetées — l'utilisateur les corrige dans l'UI.	Infra
8	Persistance Chaque transaction est sauvegardée via le repository. Un ID unique est généré (ex. REC-250122-00001) avant l'INSERT. Pour les fichiers .txt, les items d'action ([todo]/[done]) sont extraits et persistés séparément dans action_items. L'ID est conçu pour être parlant sur le type de fichier, la date d'enregistrement yymmdd et numéro du fichier enregistré (un increment)	Infra / DB

9	Mise à jour des statistiques Après persistance, <code>uploaded_files.tx_count</code> et <code>uploaded_files.total_amount</code> sont mis à jour en une seule requête. Ces valeurs alimentent directement la vue Fichiers.	Infra / DB
10	Réponse IngestionResult Le service retourne un <code>IngestionResult</code> par fichier : statut (ok / partial / ocr_failed / no_parser), <code>tx_count</code> , <code>total_amount</code> , et la liste d'erreurs. Le routeur collecte un résultat par fichier et retourne la liste.	API

Pipeline CRUD (lecture / écriture standard)

Pour toutes les opérations qui ne font pas appel à l'ingestion de fichiers (GET /transactions, POST /invoices, PATCH /sources, etc.), le flux est simplifié : HTTP → Validation Pydantic → Injection de dépendances (Depends()) → Service → Repository SQL → Dataclass → Schéma Pydantic Read → JSON. Aucune logique métier ne réside dans le routeur.

3. Choix méthodologiques et d'outils

3.1 Parsing de fichiers — pourquoi des bibliothèques classiques ?

Le choix de s'appuyer sur des bibliothèques de parsing classiques plutôt que sur des modèles de langage ou de vision est délibéré et motivé par deux raisons complémentaires.

1. Contrainte matérielle.

Faire tourner un modèle d'OCR neuronal (TrOCR, Donut, PaddleOCR-v3) ou un LLM de vision localement exige un GPU dédié. En l'absence de puissance de calcul suffisante, opter pour des modèles cloud aurait introduit une dépendance payante que les règles du challenge interdisent explicitement.

2. Choix académique.

Une part significative de la recherche doctorale porte précisément sur l'extraction et le nettoyage de données non structurées ainsi que sur le prompt engineering avec études d'ablation. Utiliser des LLMs pour le parsing aurait constitué un défi moindre. Concevoir un pipeline robuste avec des outils classiques (et en mesurer les limites) est plus révélateur des décisions d'ingénierie réelles.

PyMuPDF (fitz) pour PDF et images

Bibliothèque choisie pour le parsing des PDF (relevés de carte, factures) et la conversion d'images haute résolution en texte via son moteur OCR intégré (basé sur Tesseract). Avantages clés par rapport aux alternatives :

- Extraction de texte native pour les PDF numériques (pas de rendu en image intermédiaire). Il est bien plus rapide et précis que *pdfplumber* sur des layouts tabulaires complexes.
- **Support natif des images embarquées dans les PDF** : PyMuPDF extrait et rasterise les figures sans passer par une bibliothèque tierce.
- **API unifiée** : le même objet `fitz.Document` gère PDF, PNG, JPEG, TIFF, BMP. Un seul parser suffit pour les reçus image et les relevés PDF.
- Très faible empreinte mémoire pour des fichiers volumineux grâce au chargement paresseux page par page.
- Maintenu activement (MIT licence), bien documenté, binaire pré-compilé sur toutes les plateformes. Son installation triviale.
- Alternative écartée : *pdfplumber* (moins fiable sur des colonnes financières irrégulières) ; *camelot* (dépendance Ghostscript lourde).

pytesseract + Pillow pour OCR sur reçus photographiés

Pour les reçus photographiés (JPG/PNG), PyMuPDF délègue le rendu vectoriel mais pytesseract assure la reconnaissance de caractères via le moteur Tesseract-OCR local. Pillow pré-traite l'image avant la reconnaissance (binarisation, redressement, augmentation de résolution).

- Tesseract est l'état de l'art open-source de l'OCR ; son modèle LSTM (4.x) atteint 97 %+ sur du texte imprimé propre.
- Pillow permet des pipelines de pré-traitement arbitraires (contraste, netteté, skew) sans dépendance OpenCV.
- La combinaison est entièrement offline, sans quota et reproductible.
- Limite identifiée : les reçus froissés ou mal éclairés font chuter la précision. Un score de confiance OCR est calculé et stocké (`ocr_confidence`) pour alerter l'utilisateur.

openpyxl pour relevés XLSX et factures tableur

Les relevés de carte de crédit et les tableurs de facturation exportés en `.xlsx/csv` sont parsés via `openpyxl`, sans passer par `pandas`.

- Lecture directe des cellules sans chargement de l'ensemble du `DataFrame` en mémoire performant sur de gros relevés.
- Accès aux métadonnées de formatage (couleur de cellule, bordures) utiles pour détecter les séparateurs de sections dans certains relevés bancaires.
- Zéro dépendance transitive lourde (pas de NumPy requis).

- Alternative écartée : pandas aurait simplifié certaines transformations mais ajouté ~20 Mo de dépendances pour un usage ponctuel.

Parser TXT maison pour les notes et items d'action

Les fichiers notes.txt suivent une convention simple ([todo] / [done] / texte libre). Un parser léger en pur Python extrait les items d'action sans aucune bibliothèque externe. Les lignes [todo] deviennent des ActionItem ouverts ; les lignes [done] sont marquées terminées ; les autres lignes non structurées sont conservées comme contexte.

3.2 API — FastAPI

FastAPI a été retenu comme framework API pour plusieurs raisons structurelles :

- Gestion native de l'asynchronisme : les endpoints peuvent être async ou sync selon le besoin. Pour un projet qui pourrait évoluer vers des webhooks bancaires ou des WebSockets temps réel, cette dualité est essentielle sans refactoring.
- Validation automatique des entrées/sorties via Pydantic v2 : chaque payload entrant est vérifié, converti et documenté automatiquement. Les erreurs de type remontent avec des messages exploitables avant d'atteindre la logique métier.
- Documentation OpenAPI générée automatiquement (Swagger UI sur /docs, ReDoc sur /redoc) . Utile pour tester les endpoints pendant le développement et pour la livraison au challenge.
- Faible latence grâce à Starlette sous-jacent et à uvicorn (serveur ASGI) : temps de réponse inférieurs à 5 ms sur les requêtes CRUD locales.
- Dependency Injection (Depends()) propre : la session SQLAlchemy, les repositories et les services sont construits et nettoyés automatiquement à chaque requête sans état global.
- Scalabilité : en production, il suffit d'augmenter le nombre de workers uvicorn ou de passer devant un load balancer. Aucun changement de code.

3.3 Base de données — SQLite + SQLAlchemy 2.x

SQLite est un choix de démarrage, pas de compromis

SQLite a été choisi pour éliminer toute friction d'installation. Pas de serveur à démarrer, pas de credentials à configurer, pas de port à ouvrir : le projet est opérationnel après make install. Pour un challenge de 48 heures, c'est une décision économique et correcte.

- Mode WAL (Write-Ahead Logging) activé dès l'initialisation : lectures concurrentes sans blocage, écritures sérialisées sans erreurs.

- Le fichier de base de données (data/shoebox.db) est totalement portable — une copie suffit à transporter l'état complet de l'application.
- En production, basculer vers PostgreSQL ou MySQL ne requiert qu'un changement de ligne dans .env (DATABASE_URL). SQLAlchemy génère le DDL et le dialecte SQL appropriés. Aucun code applicatif ne change.

SQLAlchemy pour le découplage du SGBD

- Abstraction complète du dialecte SQL : les requêtes sont écrites une fois et fonctionnent identiquement sur SQLite, PostgreSQL et MySQL.
- Mapped / mapped_column (style 2.x) : définition des modèles ORM en Python pur avec annotations de type, sans magie déclarative.
- Sessions scope-per-request via get_session() : isolation propre, pas d'état global, rollback automatique en cas d'exception.
- Les IDs sont générés en Python (generate_id()) avant l'INSERT, pas de SEQUENCE SQL spécifique à un SGBD. Le format PREFIX-YYMMDD-NNNNN est lisible et traçable.
- Le fichier de configuration est backend/core/config.py (classe Settings héritant de BaseSettings). La variable DATABASE_URL est l'unique point de changement pour migrer de SGBD.

Pydantic pour le typage fort à la frontière API

- Chaque payload entrant (TransactionCreate, InvoiceCreate, etc.) est validé par un schéma Pydantic avant d'atteindre le service. Les erreurs de type, de plage ou de format remontent avec HTTP 422 et un message précis.
- Chaque réponse (TransactionRead, IngestionResult, etc.) est sérialisée par Pydantic. Et aucune donnée inattendue ne fuit vers le client.
- Séparation stricte des trois types de modèles : dataclasses domaine (core/models.py), ORM SQLAlchemy (orm_models.py), schémas Pydantic (schemas/). Chaque couche a sa propre représentation des données.

3.4 Frontend en Dash / Plotly

Dash a été préféré à une stack JavaScript (React, Vue, Svelte) pour plusieurs raisons qui vont au-delà de la simple commodité :

- Cohérence du stack Python à bout en bout : le même développeur qui écrit les services FastAPI écrit les vues Dash. Pas de context-switch entre deux langages, deux gestionnaires de paquets, deux paradigmes de composants.

- Accès direct aux données Python : les callbacks Dash manipulent des dicts et des listes Python natifs retournés par l'API client. Pas de sérialisation/désérialisation JSON supplémentaire dans le code frontend.
- Plotly sous-jacent : les graphiques (barres groupées, barres empilées, camemberts) sont définis en Python avec des objets `go.Bar`, `go.Pie` est réutilisables, testables unitairement, sans JavaScript.
- Pattern-matching callbacks (ALL, MATCH) : Dash permet des callbacks dynamiques sur des composants générés à la volée (formulaires de factures, onglets de filtre) sans écrire de logique DOM complexe.
- Productivité sur 48 heures : un composant Dash complet (layout + callback) prend 30 à 50 lignes de Python. L'équivalent React aurait nécessité hooks, state management, fetch, et rendu conditionnel séparément.
- Déploiement simplifié : l'application Dash expose un serveur Flask sous-jacent (`app.server`). Un gunicorn devant suffir pour un déploiement production. Même commande que le backend FastAPI.

3.5 Tests avec pytest et avec isolation SAVEPOINT

Les tests unitaires couvrent le backend (services, parsers, catégorisation, router transactions). Chaque test s'exécute dans un SAVEPOINT SQLAlchemy : même si le code testé appelle `session.commit()`, toutes les écritures sont annulées à la fin du test. Résultat : isolation totale, base en mémoire, pas de fichiers temporaires, exécution < 2 secondes.

- `conftest.py` fournit une session en mémoire et des repositories pré-instanciés via fixtures pytest.
- Les tests de router utilisent `httpx.AsyncClient` sur l'app FastAPI sans démarrer de serveur réseau réel.
- La couverture porte sur la catégorisation, le générateur d'IDs, le service d'ingestion et les routes CRUD de transactions.

4. Limites actuelles et pistes d'amélioration

4.1 Parsing piloté par LLM

Le pipeline de parsing actuel repose sur des règles et des expressions régulières. Un changement de mise en page dans un relevé bancaire ou une facture exige de modifier manuellement les règles — et risque d'introduire des régressions sur les formats déjà gérés.

Avec plus de temps : un LLM (Claude Haiku ou un modèle open-source léger comme Phi-3) servirait de couche de validation après le parsing classique. Il détecterait les incohérences (montant vs texte, date ambiguë, TVQ manquante) et suggérerait une correction structurée avant la persistance. Le parsing classique resterait le chemin principal — rapide et déterministe — et le LLM interviendrait uniquement en cas de score de confiance faible.

4.2 Détection de doublons par similarité

La déduplication actuelle repose sur le hash SHA-256 du fichier. Un premier upload échoué mais partiellement enregistré, ou deux fichiers légèrement différents contenant les mêmes transactions, passent à travers le filet.

Avec plus de temps : une empreinte vectorielle des transactions (date + montant + description encodée) permettrait de détecter les doublons par similarité cosinus. Avant chaque import, le système comparerait les nouvelles transactions avec l'historique et enverrait une alerte à l'utilisateur si un fichier similaire a déjà été intégré avec succès.

4.3 Système d'alertes proactif

L'application est actuellement réactive : l'utilisateur voit les anomalies quand il ouvre le dashboard. Un système proactif enverrait :

- Un récapitulatif hebdomadaire par email ou Slack (factures en retard, dépenses personnelles sur carte pro, pic de dépenses).
- Des alertes immédiates dès qu'une anomalie est détectée à l'ingestion (doublon probable, montant suspect, date incohérente).

Les interfaces IParser et les services sont conçus pour accueillir un NotificationService sans modifier le code existant. Il s'agit d'une extension, pas une refonte.

4.4 Suppression en cascade avec traçabilité de source

Actuellement, il n'est pas possible de supprimer l'ensemble des transactions issues d'un fichier particulier une fois celui-ci ingéré. Supprimer un fichier ne supprime pas ses transactions en base. C'est une faiblesse fonctionnelle importante.

La correction requiert que chaque ligne de transaction soit explicitement liée à son `uploaded_file_id` source. La colonne existe partiellement, mais la contrainte de clé étrangère et la route DELETE `/files/{id}` avec cascade ne sont pas implémentées. Avec plus de temps, cette suppression en cascade aurait été complète : fichier → transactions → items d'action associés.

4.5 Migrations de schéma (Alembic)

Les tables sont créées via `create_all_tables()` au démarrage. Si le schéma évolue (nouvelle colonne, nouvelle table), il faut actuellement supprimer et recréer la base. Alembic était listé comme dépendance mais n'a pas été intégré faute de temps. Une prochaine version ajouterait des migrations versionnées et réversibles.

4.6 Design et documentation du code

Le design system frontend (couleurs, typographie, espacements) est centralisé dans frontend/theme.py. C'est cohérent et maintenable. Sur l'interface elle-même, des améliorations restent possibles : états de chargement plus granulaires, messages d'erreur contextuels, mode responsive pour tablettes.

La documentation est complète sur les couches critiques (services, parsers, repositories). Certaines sous-composantes du frontend (callbacks complexes, helpers de formulaires) mériteraient des docstrings plus détaillés sur les invariants attendus.

4.7 Ce qu'on aurait pu faire en plus

Connexion bancaire directe (Plaid / Flinks)

Remplacer l'import manuel de relevés par une connexion OAuth au compte bancaire. Les transactions arriveraient automatiquement chaque nuit via un cron APScheduler. Le BankSyncService passerait par le même IngestionService existant. Seul le parser changerait (BankTransactionParser vs XLSXParser).

Export fiscal T2125 (Revenu Québec / ARC)

Générer automatiquement le formulaire T2125 (État des résultats d'une entreprise) pré-rempli à partir des catégories Shoebox. openpyxl et le template ARC suffiraient. Les données sont déjà structurées et catégorisées.

Multi-utilisateur & authentification

L'architecture est isolée par requête (pas d'état global) mais toutes les données sont partagées. Ajouter un user_id sur chaque table + JWT (FastAPI Security) transformerait Shoebox en SaaS sans refonte de la logique métier.

Docker / docker-compose

Un Dockerfile multi-stage (builder Python + image finale slim) et un docker-compose.yml (backend + frontend + volume data/) auraient simplifié le déploiement à un seul docker compose up.

5. Principes SOLID appliqués

Principe	Application dans le code
SRP — Single Responsibility	Chaque service a un rôle unique : IngestionService orchestre, TransactionService gère le CRUD, AnalyticsService calcule les KPIs. Les parsers parsent. Les repositories persistent.
OCP — Open/Closed	Ajouter un format de fichier = ajouter une classe implémentant IParser. Aucun code existant ne change. Même logique pour les règles de catégorisation (liste RULES).
LSP — Liskov Substitution	Tous les parsers sont interchangeables via IParser. L'IngestionService appelle can_parse() et parse() sans connaître la classe concrète.
ISP — Interface Segregation	Six interfaces de repository distinctes (ITransactionRepository, IInvoiceRepository, ISourceRepository, IFileRepository, IAnomalyRepository, IActionRepository) au lieu d'une grande interface fourre-tout.
DIP — Dependency Inversion	Les services dépendent d'interfaces (ABCs), jamais de classes concrètes. FastAPI Depends() câble les implémentations concrètes au moment de la requête.